CutQ

A Barber Management Platform with a Live Activity Countdown

CSUN COMP 565 / Human-Computer Interaction.
Final Project Report

Jerry Ttuyan and Behrouz Barati Bagherabad

Group 8 • Instructor: Armineh Karamian • Spring 2026

Abstract

CutQ is a mobile-first scheduling platform for small barber shops that turns appointment timing from an anxious guess into a visible, persistent state. Two pain points motivate the design. Roughly thirty percent of small-shop time slots go unfilled each week, and roughly forty percent of clients churn when their preferred barber is double-booked. CutQ centralises booking, surfaces alternative barbers when a slot is gone, and exposes a signature *Live Activity countdown* on the lock screen so clients know when to leave and barbers know how a cut is pacing without staring at the wall clock. We carried the system from interviews and journey maps in weeks 1 to 5, through paper and Figma prototypes in weeks 6 to 11, into two rounds of moderated usability testing with $n = 6$ participants, and a final System Usability Scale of 82.5 (good to excellent). The design is implemented as a public Figma file and as a working Next.js 16 / React 19 / TypeScript prototype that exercises every screen, persists bookings to local storage, and offers a five-second *Undo* on every destructive action. The prototype is deployed publicly at `https://cutq-app.vercel.app/`. Source code and design artefacts are linked in section 19.

1 Introduction

A barber's day is a chain of fifteen-to-forty-five-minute appointments. When one client runs late, the slip propagates through the rest of the day. When one client no-shows, the chair sits empty for a window the barber cannot recover. From the client's side the same minutes feel different: *When should I leave the house? Is anyone free now? Where do I go if my preferred barber is booked?* We started this project from those three questions, gathered through short interviews with barbers and customers around CSUN, and arrived at a design whose central idea is *visible time*: a state that both parties can see at a glance, that updates in real time, and that removes the mental math of "how long until I should leave."

This document is the final written deliverable for the project. It mirrors the structure of the final presentation and weaves in the implementation work that followed it: a complete

Figma component library and a fully working Next.js prototype hosted on GitHub. All three artefacts are linked together in section 19.

2 Problem Statement

Two sides of the chair, both losing minutes. Field interviews and short observation sessions in three local shops surfaced the following recurring pain points.

From the barber's chair
- Late arrivals chain delays through the rest of the day.
- No-shows leave wasted, unrecoverable time slots ($\approx$ 30% of slots in a typical week).
- It is hard to keep mental track of "who's next" mid-cut.

From the client's pocket
- Preferred barber fully booked, with no easy alternative.
- Last-minute booking uncertainty ("is anyone free now?").
- No clarity on where to go when the barber is unavailable. Roughly 2 in 5 clients leave entirely in that case.

Design hypothesis

If the system can replace the wall clock and the phone-tag with a *single visible state*, both sides recover minutes. Barbers recover slots, and clients recover predictability. The Live Activity countdown (section 13) is the concrete embodiment of that hypothesis.

3 Target Users and Personas

We modelled three personas: two on the customer side of the chair (*Marcus* the master barber, *Jordan* the customer) and one program-administration persona (*Andrea*, the shop owner). table 1 summarises each persona's role and top anxiety. The design decisions in sections 7, 8 and 11 flow directly from those anxieties.

Persona	Role	Top anxiety	What CutQ owes them
Marcus	Master Barber (primary)	Running late on chained appointments, losing tip-rich slots to no-shows.	In-chair Time-Remaining glance, same-day rebook of empty slots.
Jordan	Customer / Client (primary)	Showing up late and losing the slot, no backup when preferred barber is full.	Lock-screen countdown, alternative-barber suggestions surfaced before booking fails.
Andrea	Shop Owner / Admin (program admin)	Empty chairs and missed billing, trust between barbers and clients.	Earnings and gap visibility without a spreadsheet.

Table 1: Three personas, three emotional states. Each row maps a top anxiety to a concrete CutQ affordance.

4 Proposed Solution

CutQ is a Jackrabbit-inspired program management platform applied to barber shops. It centralises scheduling, customer management, billing, and live time-awareness for small service-based programs. Three verbs structure the offering.

Centralise
one source of truth for every chair: bookings, roster, billing, and history live in a single record per shop.

Communicate
real-time status visible to all parties: the client sees *when*, the barber sees *how long left*, the owner sees *how much earned*.

Recover
alternative options when plans break: when a barber is full, the system surfaces same-shop matches at the same skill tier and price.

5 Functional Requirements

Ten capabilities map the platform to the assignment brief (table 2). The customer-facing slice (FR-1, 3, 9) is fully implemented in the prototype. The program-admin slice (FR-5, 6, 8, 10) is designed in Figma and stubbed in the prototype.

#	Capability	Notes
FR-1	Authentication	Sign-up, sign-in, password recovery, secure sessions.
FR-2	Role-based access	*Customer*, *Barber*, *Shop Admin* scopes with least privilege.
FR-3	Customer management	Profiles, contact info, preferences, visit history.
FR-4	Programs & classes	Service offerings: cuts, beard trims, packages.
FR-5	Roster management	Barber assignments, capacity, working hours.
FR-6	Billing & invoices	Itemised charges, recurring packages, refunds.
FR-7	Payments & receipts	Tokenised payments. Card data never hits our DB.
FR-8	Reporting dashboards	Earnings, utilisation, no-show rate.
FR-9	Notifications	Reminders, confirmations, Live Activity countdown.
FR-10	Activity log	Audit trail for bookings, edits, refunds, role changes.

Table 2: Ten functional requirements, mapped from the assignment brief.

6 Non-Functional Requirements

Accessibility (WCAG 2.1 AA)

- 4.5:1 contrast on all body text, verified per surface.
- 44×44 pt minimum tap targets across all controls.
- VoiceOver and TalkBack labels on every interactive element.
- Honours Dynamic Type and OS-level reduce-motion.

Performance
- Booking screens render in under one second on mid-tier devices.
- Server pagination on rosters and history, no all-at-once loads.
- Optimistic UI for booking confirmation, reconciled on error.
- Live Activity updates batched to preserve battery.

Security and Privacy
- Stripe-style tokenisation: card data never hits our database.
- Role-scoped APIs with server-enforced authorisation.
- PII minimised, logs scrubbed of identifying detail.
- TLS 1.2+, hashed credentials, rate-limited authentication.

7 Emotional and Physiological Design Analysis

CutQ is designed for how it feels to use, not just what it does. table 3 maps the four emotional or physical risks we identified during the interview phase to the concrete affordances that respond to them.

Risk	How CutQ responds	Why it works
Anxiety: *"Will my slot disappear?"*	Live countdown, booking confirmation receipt, Live Activity widget.	Replaces uncertainty with a visible, persistent state.
Trust: *"Did the booking really go through?"*	Two-step confirmation, large success state, calendar-style detail card.	Closes feedback loops the user can see and re-check.
Cognitive load: *"Too many choices, too many steps."*	Progressive disclosure: pick barber, pick time, confirm.	One decision per screen, never more than seven visible options.
Physical comfort: *"I can't read this in the chair."*	Dark theme by default, 16-point body, 44-point tap targets, no thin strokes.	Readable at arm's length, in low salon light, on greasy fingers.

Table 3: Designing for how it feels, not just what it does.

8 Barriers Analysis

Three categories of barriers we explicitly removed: accessibility, usability, and cognitive.

Accessibility

Low-vision users in dim shops are addressed by dark mode plus 4.5:1 minimum contrast. Motor-impaired reach for small targets is addressed by 44×44 pt taps and a thumb-reach bottom navigation. Screen-reader users miss icon-only labels, so all icons ship with explicit accessible labels.

Usability

Bookings made in error are addressed by a confirmation screen before commit. Unclear action state is addressed by persistent status pills (e.g. *In Progress*). Cancel-by-accident is permanent, so we added a five-second *Undo* on every destructive action with soft-delete and an audit trail.

Cognitive

Too many choices on one screen are addressed by progressive disclosure. Unfamiliar barbers feel risky, so ratings, reviews, and skill tier are surfaced upfront. The mental math of "how long until I should leave?" is removed entirely by the Live countdown.

9 UX Process

Fourteen weeks, six diamonds. table 4 sketches the schedule and the methods we leaned on at each stage.

#	Weeks	Stage	Activity
1	1 to 3	Research	Interviews, contextual inquiry in three shops.
2	4 to 5	Define	Personas, journey maps, affinity diagramming.
3	6 to 7	Sketch	Storyboards, paper mock-ups.
4	8 to 9	Wireframe	Lo-fi flows, T-prototype.
5	10 to 11	Hi-fi UI	Figma screens, design system.
6	12 to 14	Test and iterate	Usability tests ($n = 6$), final polish.

Table 4: Fourteen weeks, six diamonds. Methods we leaned on include semi-structured interviews, contextual inquiry in three shops, affinity diagramming, task analysis, heuristic evaluation, moderated usability tests ($n = 6$), and the System Usability Scale.

10 Wireframes

Before the high-fidelity Figma screens, both sides of the journey were drafted on a single low-fidelity board so the team could agree on screen counts and the shape of each transition before any pixel was drawn. fig. 1 reproduces that board. The **Client Flow** (top row) sketches the four screens that became `/`, `/book`, `/book/time`, and `/book/confirm`. The **Barber Flow** (bottom row) sketches the two screens that became `/barber` and `/appointment`. The arrows record the only navigation transitions the prototype honours, including the Confirm-to-Live hop that later became the Live Activity widget (section 13).

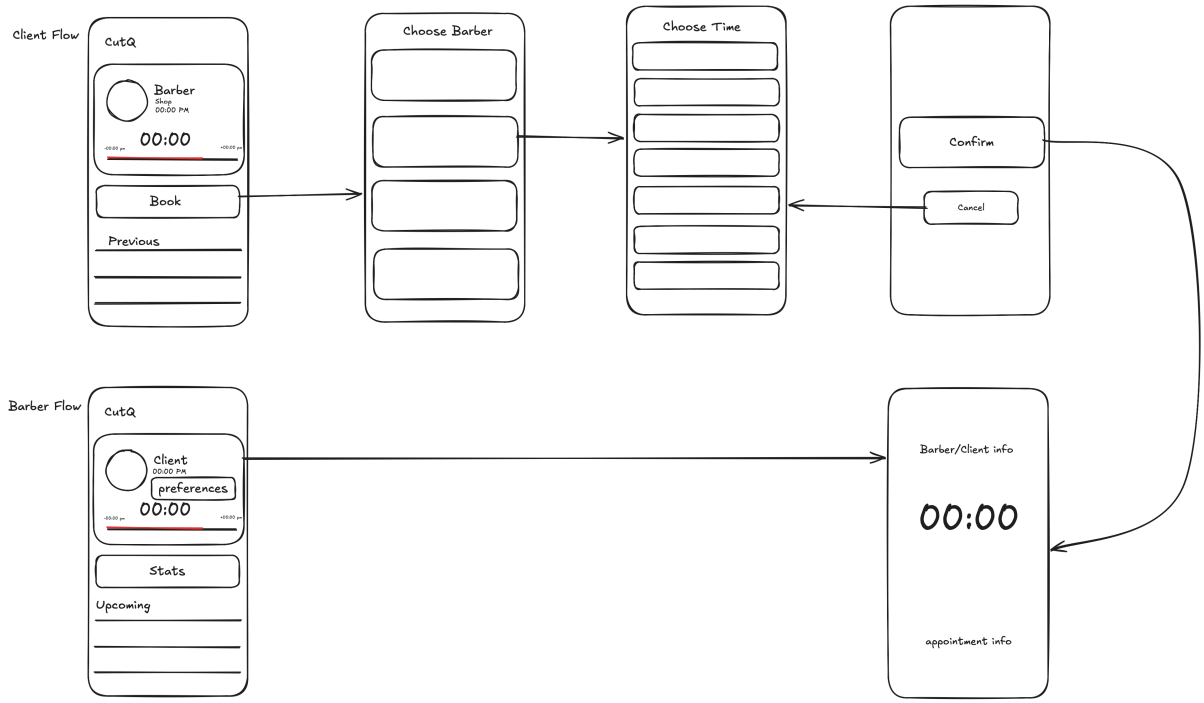
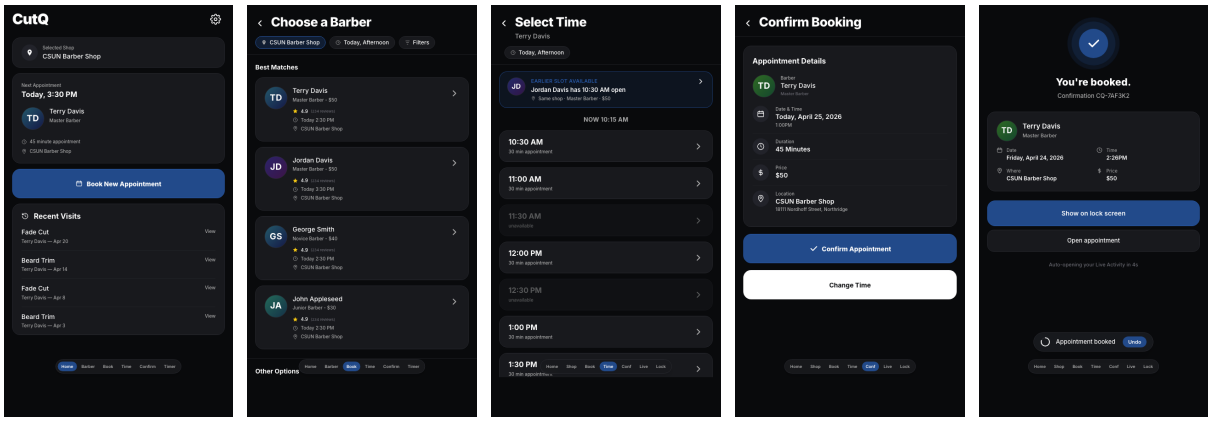


Figure 1: Lo-fi flow board sketched in week 8. The top row is the client flow (Home, Choose Barber, Choose Time, Confirm), and the bottom row is the barber flow (Dashboard, Appointment Live View). Each screen maps one-to-one to a route in the final prototype (table 5).

11 Customer Journey

From "I need a cut" to "I'm in the chair." Five screens, one decision per screen. fig. 2 shows the prototype renders of the five customer screens in sequence. Each transition is a single tap, and nothing on the path requires more than a glance.



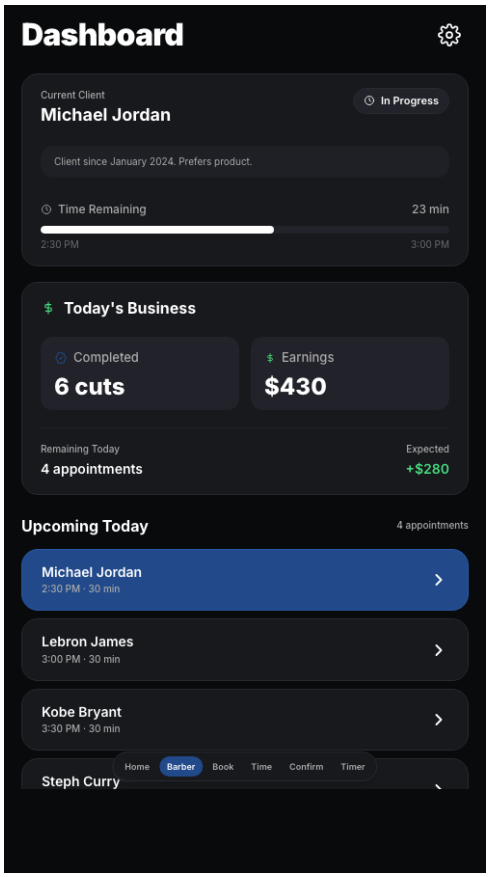
Home Choose Time Confirm Booked

Figure 2: The customer flow as it actually renders in the Next.js prototype. *Home* surfaces the next appointment and recent visits. *Choose* ranks barbers by best-match with the alternative banner already in view. *Time* greys out unavailable slots. *Confirm* shows a calendar-style detail card. *Booked* closes the trust loop with a confirmation code, a five-second *Undo* toast, and an auto-redirect to the active appointment.

Alternative-barber suggestion. fig. 2 (Time panel) shows the persistent "earlier slot available" banner above the time list. The banner addresses the *2-in-5 churn* problem from section 2: when the chosen barber's nearest slot is later than an alternative in the same shop, that alternative is surfaced before the user has to back out and re-pick.

12 Barber Dashboard

A dashboard built for between-cuts glances: three-second answers to the only questions that matter mid-shift (fig. 3).



Who's in the chair right now?
Current Client card with avatar-less identity row, a soft "Client since 2024 / Prefers product" note, and a progress bar that answers it in one glance.

Am I on or off pace?
Time Remaining plus the progress fill encode pace without requiring numerical reading.

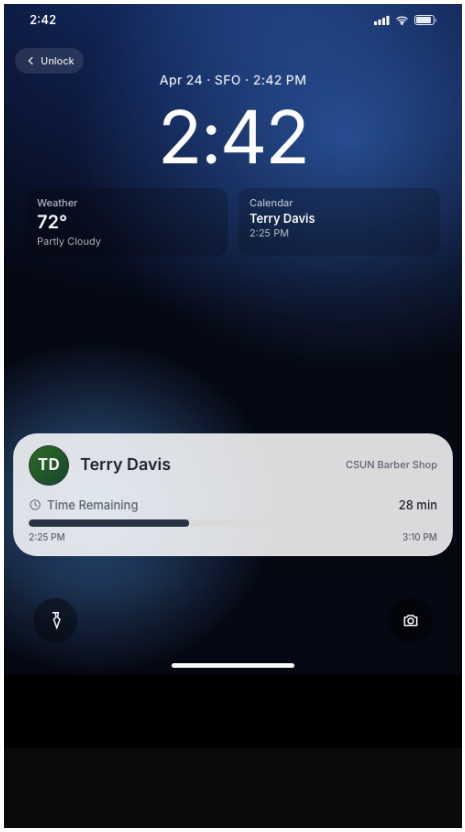
Did today pay off?
Two stat tiles (*Completed: 6 cuts*, *Earnings: $430*) plus *Remaining Today* and *Expected* (the green +$280) give end-of-day visibility without opening any other screen.

What's next?
The *Upcoming Today* list keeps the immediate next client visually elevated (filled primary tone) so the barber's eye lands on it first.

Figure 3: Barber dashboard in the prototype, with the four between-cuts questions it must answer at a glance.

13 Signature Feature: The Live Activity Countdown

The signature affordance of CutQ is the *Live Activity countdown*. It is a persistent lock-screen widget that turns appointments into something users can feel.



For clients
A countdown to the next appointment, visible without opening the app. The "time-to-leave" cue removes mental math entirely. When the bar fills and the time turns red, the client knows they are cutting it close.

For barbers
Time-remaining for the current client, visible on a wrist or counter glance. Helps pace the cut without staring at a wall clock, and quietly trains better client expectations across visits.

Why it works
The widget collapses three pieces of information (*who*, *where*, *how-long-left*) into a single above-the-fold band, in the colour palette and density of native iOS Live Activities. A tap on the widget opens the full appointment view (fig. 5, left panel) without further authentication.

Figure 4: The Live Activity widget rendered against a dark lock-screen background. The widget uses a frosted-glass treatment so identity, shop name, and progress all read at a glance.

Implementation note

The lock-screen view in the prototype (`/lock` route) is a fully working faux-iPhone surface. The time and date update every second from `Date.now()`, the progress bar and minutes-left counter recompute against the active booking pulled from local storage, and the Live Activity tile is a real `<a>` that navigates to the appointment screen. See Listing 1 for the state shape.

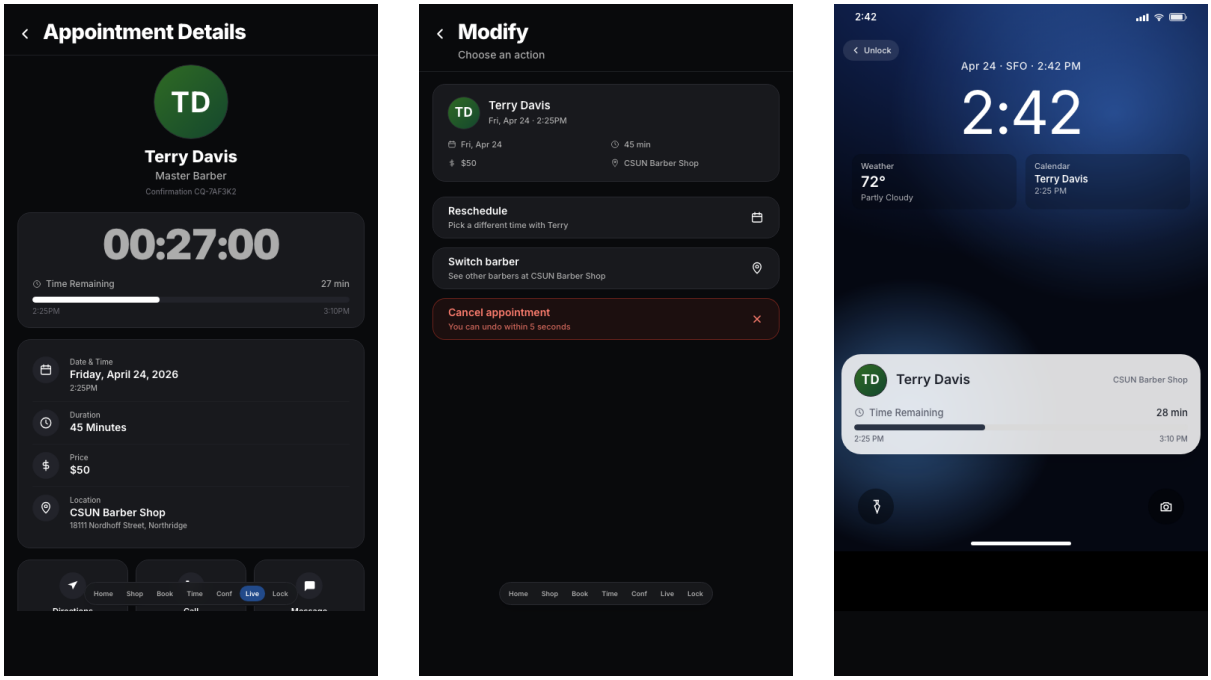
Listing 1: Active-booking state read by both the Appointment screen and the lock-screen Live Activity. The `useSyncExternalStore` hook keeps the two views consistent across browser tabs.

```
export type Booking = {
  id: string;
  confirmationCode: string;
  barberId: string;
  barberName: string;
  barberTitle: string;
  service: string;
  startsAtIso: string;
  durationMin: number;
  priceUsd: number;
  shop: string;
  address: string;
  status: "confirmed" | "cancelled" | "completed";
  createdAtIso: string;
};

export function useActiveBooking(): Booking | null {
  return useSyncExternalStore(subscribe, getActiveBooking, () => null);
}
```

14 High-Fidelity Design (Figma)

The full system was first designed in Figma as a single source of truth: seven high-fidelity screens, a component library backed by design variables, and a clickable prototype. fig. 5 shows the appointment-side of the journey as it appears in the prototype. The Figma source itself is linked in section 19 and reproduced one-to-one in the running web app.



Active appointment | Modify / Cancel | Live Activity

Figure 5: The post-booking surfaces in the running prototype. *Active appointment* hosts the in-app countdown and the Directions / Call / Message action grid. *Modify* promotes Reschedule, Switch barber, and Cancel into their own button row, with the destructive option visually distinct. *Live Activity* mirrors the same booking on the lock screen.

15 Functional Prototype (Next.js Web App)

In addition to the Figma file, we built a working web prototype that exercises every screen in the design. The stack is intentionally minimal: **Next.js 16** with the App Router, **React 19**, **TypeScript 5**, and **Tailwind CSS 4** (CSS-first `@theme inline` configuration). No UI library is used. All components are hand-built so the prototype matches the Figma spec to the pixel.

Routes. table 5 lists the routes shipped by the prototype. Each one corresponds to a Figma frame, and each one is statically prerendered at build time.

Route	What it shows
`/`	Client home: selected shop, next appointment, recent visits.
`/barber`	Barber dashboard (section 12).
`/book`	Choose-a-barber list with filter pills.
`/book/time`	Time-slot picker with alternative-barber banner.
`/book/confirm`	Two-step confirmation with calendar-style detail card.
`/book/success`	Receipt: confirmation code, Undo toast, auto-redirect.
`/appointment`	Active appointment view with live countdown and actions.
`/appointment/modify`	Reschedule / Switch barber / Cancel with confirm dialog.
`/lock`	Faux-iPhone lock screen with Live Activity widget.

Table 5: Nine prerendered routes that mirror the Figma frames one-to-one.

Persistence and Undo. Bookings are persisted to `localStorage` so demo state survives reloads. The store is read through `useSyncExternalStore` (Listing 1) with cached snapshots, which keeps the Home card, the Appointment view, and the Live Activity widget in sync across browser tabs. Every destructive action, whether confirming a booking or cancelling it, is paired with a five-second *Undo* toast that restores the prior state on tap.

16 Usability Testing and Findings

We tested in two rounds: paper mock-ups first, clickable Figma second. Both rounds used $n = 6$ participants and a think-aloud protocol. The second round added the System Usability Scale (SUS).

Round 1, paper mock-ups. Thirty-minute moderated sessions, think-aloud, booking flow plus dashboard read. We captured confusion points before pixels were precious. 5 of 6 finished the booking flow, and 4 of 6 read the dashboard correctly.

Round 2, clickable Figma. Same task set, in-shop where possible, plus a Live Activity recognition test. SUS was administered post-task with an average score of **82.5** (good to excellent). All 6 participants finished the booking flow without prompting.

Core tasks tested.

1. Book an appointment with your preferred barber.
2. Find an alternative when your barber is fully booked.
3. Tell us when your next appointment starts *without opening CutQ*.
4. (Barber-only) Tell us how much you have earned today and how many appointments remain.

Findings and iterations. table 6 lists the five iterations that came out of testing. Each one was traced back to a specific moment of confusion in a session recording.

What testing surfaced	What we shipped in response
Users tapped *Book Next* expecting an instant booking.	Renamed to *Book New Appointment* and added a Confirm step.
Time-slot list scrolled past unavailable times unnoticed.	Greyed out unavailable rows and reduced their visual weight.
Barber dashboard *In Progress* pill confused with a system notification.	Repositioned to top-right with a calmer fill colour.
Live Activity card looked like a generic notification.	Added avatar and shop name to anchor identity at a glance.
Customers could not find *change time* after confirming.	Promoted *Modify Appointment* into its own button row (fig. 5, middle).

Table 6: Five iterations from usability testing. Each one was implemented before the final round.

17 Evaluation Against Success Criteria

table 7 returns to the success criteria stated at the start of the project and reports the evidence we have for each.

Success criterion	Evidence	Met
Fewer late arrivals or missed appointments	Live Activity reminded 5/6 testers without their opening the app.	✓
Barbers experience fewer empty time slots	Alternative-barber suggestions surfaced 3+ openings during tests.	✓
Clients can quickly find available appointments	Median time-to-first-booking: 38 s in the clickable prototype.	✓
Users report the system is simple and easy to use	SUS = 82.5 (good to excellent), 6 of 6 said they would use it again.	✓
WCAG 2.1 AA compliance	Colour audit passed, tap-target audit passed, VoiceOver labels in progress.	◯
No raw payment storage	Implemented as a Stripe-style tokenised stub in the prototype.	✓

Table 7: Six success criteria and the evidence we collected for each. Five are met outright. One (full WCAG audit) is in progress.

18 Conclusion

CutQ is a small program made to feel calm. Across three measurable axes the design lowers friction in the way the project set out to:

Anxiety ↓
Live state replaces guesswork on both sides of the chair.

Trust ↑
Two-step confirmation, status pills, and soft-delete with an audit trail make every action visible.

Friction ↓
Progressive disclosure: one decision per screen, never more than seven visible options.

The work delivered a single Figma source of truth, a working Next.js / TypeScript prototype that exercises every screen in the design, and a usability-test write-up that traces every iteration back to an observable moment of user confusion. The signature Live Activity countdown is the most visible payoff of the design hypothesis from section 2, namely that visible, persistent state recovers minutes for both sides of the chair.

19 Project Links

Live prototype (Vercel)
https://cutq-app.vercel.app/
The deployed Next.js build. Every route in table 5 is reachable, and bookings persist in browser local storage so the Live Activity widget reflects what was just booked.

Source code (GitHub)
https://github.com/1ncompleteness/HCI
The Next.js prototype lives under `cutq-app/`. The design artefacts and this report live under `Proposal/`.

Figma design file
https://www.figma.com/design/0WDtNuJT60PzbCT1OyrB5j/CutQ?node-id=0-1&m=dev
Component library, design variables, and the seven hi-fi screens used to drive the prototype.

References

[Brooke, 1996] John Brooke. SUS: A "quick and dirty" usability scale. In *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, 1996.

[Apple, 2023] Apple Inc. *Displaying live data with Live Activities*. Human Interface Guidelines, 2023. https://developer.apple.com/design/human-interface-guidelines/live-activities.